

Tutor Availability Platform

with Smart Recommendation and Waitlist Management

Milestone 3 Report

Final Submission

Course: Object Oriented Programming
Institute: IBA Karachi
Teacher: Samreen Kazi

Submitted By
Daniyyal Ahmed
Adarsh Sachdev

May 2026

Contents

1	Introduction	2
2	Project Objective	2
3	Object Oriented Programming Concepts Used	2
3.1	Inheritance	2
3.2	Polymorphism	2
3.3	Encapsulation	2
3.4	Abstraction	3
3.5	Exception Handling	3
3.6	Generic and Functional Programming	3
4	System Overview	3
5	Class Descriptions	4
5.1	User	4
5.2	Tutor	4
5.3	Student	4
5.4	TimeSlot	4
5.5	Waitlist	4
5.6	Session	5
5.7	Recommendation Engine	5
5.8	Platform	5
6	Smart Features	5
6.1	Tutor Recommendation and Ranking	5
6.2	Priority Waitlist by Reliability Score	5
6.3	Automatic Slot Reassignment	5
6.4	Conflict Detection	5
6.5	Student Rating and Review System	6
6.6	Authorised Cancellation	6
6.7	Auto-Assigned IDs	6
7	Final Functionality Implemented	6
8	UML Class Diagram	6
9	Test Cases	8
10	GitHub Repository Organization	8
11	Conclusion	8

1 Introduction

The Tutor Availability Platform is an Object Oriented Programming project designed to connect students with tutors in an efficient and structured manner. The system enables tutors to create profiles, define subject expertise, manage available time slots, and handle session requests from students. This Milestone 3 report represents the final, complete version of the platform. Building upon the Milestone 2 prototype, Milestone 3 introduces a fully implemented student rating and review system, a priority-based waitlist driven by student reliability scores, authorised session cancellation, smart search by name and subject, and a graphical user interface built with Raylib. The project is split into proper .hpp and .cpp files following good software engineering practice.

2 Project Objective

The main objective is to design and implement a tutoring platform using proper Object Oriented Programming principles. The final system aims to:

- manage tutors and students through a well-structured class hierarchy
- auto-assign unique IDs to tutors, students, and time slots via static counters
- allow students to search tutors by name, subject, or both
- support tutor availability and session booking using tutor ID
- prevent double-booking through conflict detection
- manage priority waitlists when slots are occupied, ordered by student reliability
- automatically assign cancelled slots to the most reliable waiting student
- allow students to rate and review tutors after sessions
- recommend tutors by subject, ranked by student rating average
- enforce authorised cancellation only the booking student can cancel
- provide a graphical interface via Raylib in addition to the console menu

3 Object Oriented Programming Concepts Used

3.1 Inheritance

A base class User holds common attributes shared by all system users `userId`, `name`, and `contactInfo` along with their getters and setters. Both Tutor and Student inherit from User using public inheritance, acquiring these members without repeating code.

3.2 Polymorphism

`viewProfile()` is declared as a pure virtual function in User. Both Tutor and Student override it with their own implementations, displaying different information for each type. This allows a User pointer to call the correct `viewProfile()` at runtime without knowing the concrete type.

3.3 Encapsulation

All sensitive data is declared private inside every class. No external code can directly modify internal state. Controlled access is provided through public getter and setter

functions. For example, `Tutor::addRating()` is the only path to updating the rating it cannot be set directly, preventing tutors from assigning themselves inflated scores.

3.4 Abstraction

The `User` class is made abstract by declaring `viewProfile() = 0`. It cannot be instantiated directly it serves as a blueprint enforcing a contract on all derived classes. Similarly, `Platform` hides the complexity of booking, cancellation, and waitlist management behind simple public methods.

3.5 Exception Handling

Seven custom exception classes are defined in `Exceptions.hpp`: `TutorNotFoundException`, `StudentNotFoundException`, `SlotNotFoundException`, `SessionNotFoundException`, `DuplicateSlot`, `AlreadyCancelledException`, and `RatingException`. These use `try/catch/throw` to detect runtime errors and print descriptive messages without crashing.

3.6 Generic and Functional Programming

Three template functions in `Utilities.hpp` provide reusable algorithms: `linearSearch (T, Predicate)` (finds one element), `filterVec (T, Predicate)` (collects all matches), and `displayAll(T)` (accepts a function pointer). Lambdas are used throughout as predicates. `ScoreComparator` is a functor (class with `operator()`) used with `std::sort` in the recommendation engine.

4 System Overview

The system is composed of the following classes, each in its own `.hpp/.cpp` pair:

Class	Responsibility
User	Abstract base class; common identity for all users
Tutor	Tutor profile, subjects, slots, and student-driven ratings
Student	Student profile, booking history, and reliability score
TimeSlot	A tutor's time window with auto-assigned ID
Session	A confirmed booking record with status and review
Waitlist	Priority queue of waiting students ordered by reliability
Recommendation Engine	Scores and ranks tutors by subject and rating
Platform	Central controller coordinating all operations
Exceptions	Seven custom exception classes for error handling
Utilities	Template functions for generic search and display

The `Platform` class acts as the central controller. It owns vectors of `Tutor`, `Student`, `Session`, and `Waitlist` objects and coordinates all major operations. The `Recommendation Engine` is responsible for intelligent tutor ranking.

5 Class Descriptions

5.1 User

Abstract base class for all system users.

- **Attributes:** `userId`, `name`, `contactInfo`
- **Methods:** constructors, getters/setters, `viewProfile()` (pure virtual)
- **Design note:** `viewProfile() = 0` prevents direct instantiation and enforces implementation in every derived class.

5.2 Tutor

Stores tutor-specific details and manages availability.

- **Attributes:** static `nextTutorId`, `subjects`, `experienceYears`, `ratingSum`, `ratingCount`, `availableSlots`
- **Methods:** `addSubject()`, `addAvailability()`, `addRating()`, `teachesSubject()`, `isAvailable()`, `findSlot()`, `viewProfile()`
- **Design notes:** IDs are auto-assigned via static `int nextTutorId`. Rating is stored as a running average. `addAvailability()` rejects duplicate slot IDs using `std::any_of`.

5.3 Student

Stores student-specific information and tracks booking behaviour.

- **Attributes:** static `nextStudentId`, `sessionHistory`, `totalBooked`, `cancelledCount`
- **Methods:** `recordBooking()`, `recordCancellation()`, `reliabilityScore()`, `viewProfile()`, `viewSessionHistory()`
- **Design notes:** `reliabilityScore() = (totalBooked - cancelledCount) / totalBooked`. Returns 0.0 if no bookings.

5.4 TimeSlot

Represents a single available time window offered by a tutor.

- **Attributes:** static `nextSlotId`, `slotId`, `date`, `time`, `isBooked`
- **Methods:** `bookSlot()`, `freeSlot()`, `displaySlot()`
- **Design note:** static `int nextSlotId` guarantees unique IDs with zero collision risk.

5.5 Waitlist

Manages students waiting for a particular tutor slot.

- **Attributes:** `entries` (vector of `WaitlistEntry`)
- **Methods:** `addStudent()`, `getNextStudent()`, `isEmpty()`, `displayWaitlist()`
- **Design note:** This is a priority waitlist, not FIFO. Students are inserted using insertion-sort so the vector is always sorted descending by `reliabilityScore`.

5.6 Session

Represents a confirmed tutoring booking.

- **Attributes:** sessionId, tutorName, studentName, subject, slotInfo, slotId, status, isRated, studentRating, studentReview
- **Methods:** cancelSession(), completeWithRating(), displaySession()

5.7 Recommendation Engine

Provides tutor ranking for a requested subject.

- **Methods:** calculateScore(), recommendTutors()
- **Design notes:** Recommendation is by subject only no date/time filter. Score-Comparator is a functor used with std::sort.

5.8 Platform

The central controller of the entire system.

- **Attributes:** tutors, students, sessions, recommender, nextSessionId, waitlistKeys, waitlists
- **Key methods:** bookSession(), cancelSession() validates that the cancelling student is the one who booked the session. rateSession() enforces four guards. getWaitlistIndex() returns an int index instead of a reference to fix dangling pointers.

6 Smart Features

6.1 Tutor Recommendation and Ranking

Tutors are recommended by subject match and ranked by their average student rating. The subject is a hard gate.

6.2 Priority Waitlist by Reliability Score

When a slot is fully booked, students are added to a priority waitlist ordered by their reliability score (completed sessions / total booked sessions) in descending order.

6.3 Automatic Slot Reassignment

When a session is cancelled, the slot is immediately freed and the system checks the priority waitlist. If a student is waiting, the slot is automatically re-booked for the highest-reliability student.

6.4 Conflict Detection

The platform prevents double-booking. If a student tries to book an already-booked slot, they are redirected to the waitlist.

6.5 Student Rating and Review System

After a session, the attending student can submit a rating (0.0-5.0) and a written review. The tutor's average rating is updated as a running average.

6.6 Authorised Cancellation

`cancelSession(sessionId, studentId)` verifies that the student requesting cancellation is the one who actually booked the session.

6.7 Auto-Assigned IDs

Tutor, Student, and TimeSlot IDs are assigned automatically via static int counters in each class.

7 Final Functionality Implemented

The Milestone 3 final submission implements the complete platform:

- adding tutor profiles (ID auto-assigned, no self-rating)
- adding student profiles (ID auto-assigned)
- adding time slots to tutors (slot ID auto-assigned)
- viewing all tutor and student profiles
- searching tutors by name, by subject, or by both
- viewing a tutor by their ID
- recommending tutors for a subject (ranked by student rating)
- booking sessions using tutor ID and slot ID
- conflict detection with automatic waitlist redirect
- authorised session cancellation (student ID verified)
- automatic slot reassignment from priority waitlist on cancellation
- student rating and review submission after sessions
- viewing session history (live status, not stale copies)
- displaying all sessions with current status
- displaying the priority waitlist for any tutor slot
- split .hpp/.cpp file structure
- console menu interface
- graphical interface via Raylib

8 UML Class Diagram

The UML Class Diagram below represents the full architecture of the final system, showing all classes, attributes, methods, and relationships.

9 Test Cases

TC ID	Feature	Input	Expected Output
TC-01	Auto IDs	Create two Tutors, two Students, two TimeSlots	IDs are sequential (1,2,3...), no duplicates
TC-02	Book Session	Tutor ID 1, student Zara, Math, free slot	Session created, slot status changes to Booked
TC-03	Conflict Detection	Ahmed books the same already-booked slot	Ahmed added to priority waitlist, no crash
TC-04	Tutor Not Found	Book with non-existent tutor ID 999	TutorNotFoundException caught, error printed
TC-05	Student Not Found	Book with student name "Ghost"	StudentNotFoundException caught, error printed
TC-06	Slot Not Found	Book with slot ID 9999	SlotNotFoundException caught, error printed
TC-07	Cancel + Waitlist	Cancel Zara's session; Ahmed is on waitlist	Session cancelled, Ahmed auto-assigned, new session created
TC-08	Double Cancel	Cancel the same already-cancelled session	AlreadyCancelledException caught, error printed
TC-09	Unauthorised Cancel	Student B tries to cancel Student A's session	runtime_error: Authorization failed
TC-10	Cancel Missing	Cancel non-existent session ID 9999	SessionNotFoundException caught, error printed
TC-11	Recommend	Subject = "Math"	Only tutors teaching Math returned, ranked by rating

10 GitHub Repository Organization

Link: <https://github.com/daniyyalahmed767-coder/Tutor-app>

The repository is organized as follows:

- `/*.hpp` and `/*.cpp`: source files
- `main.cpp`: console entry point with 29 automated test cases
- `gui_main.cpp`: Raylib graphical interface
- `CMakeLists.txt`: builds both `platform_console` and `platform_gui`
- `README.md`: setup and compilation instructions
- `docs/`: UML diagram and milestone reports

11 Conclusion

The Tutor Availability Platform Milestone 3 delivers a complete, production-quality OOP application. Every core OOP concept inheritance, polymorphism, encapsulation, abstraction, and exception handling is demonstrated in a meaningful context, not as a formality.

Beyond the Milestone 2 prototype, Milestone 3 adds a full student rating and review system, a priority-based waitlist that rewards reliable students, authorised cancellation with student ID verification, flexible search by name and subject, auto-assigned IDs eliminating collision risk, a live session history view, and a Raylib graphical interface.